

Reto 5 — Docker, Docker Hub y despliegue en Minikube sobre Windows

Evidencias de implementación del ciclo completo de publicación y despliegue de un microservicio Cloud Native

Estudiante	Jhonatan Escobar Uribe
GitHub	https://github.com/Jhont3/micro-calc-k8s (público)
Docker Hub	https://hub.docker.com/r/jhont3/demo-micro
Repositorio base	https://github.com/gmacastil/micro-calc
Fecha	18 de julio de 2026
Módulo	Diplomado — Módulo 5 (Cloud Native / Kubernetes)

Contenido

Resumen de entregables	3
1. Verificación de prerequisites	3
2. Compilación del proyecto	5
3. Dockerfile y construcción de la imagen	5
4. Ejecución local con Docker y pruebas	6
5. Publicación en Docker Hub	7
6. Manifiestos de Kubernetes y arranque de Minikube	8
7. Despliegue en Minikube	9
8. Pruebas del microservicio en Minikube	11
9. Repositorio en GitHub	13
10. Conclusiones	13

Resumen de entregables

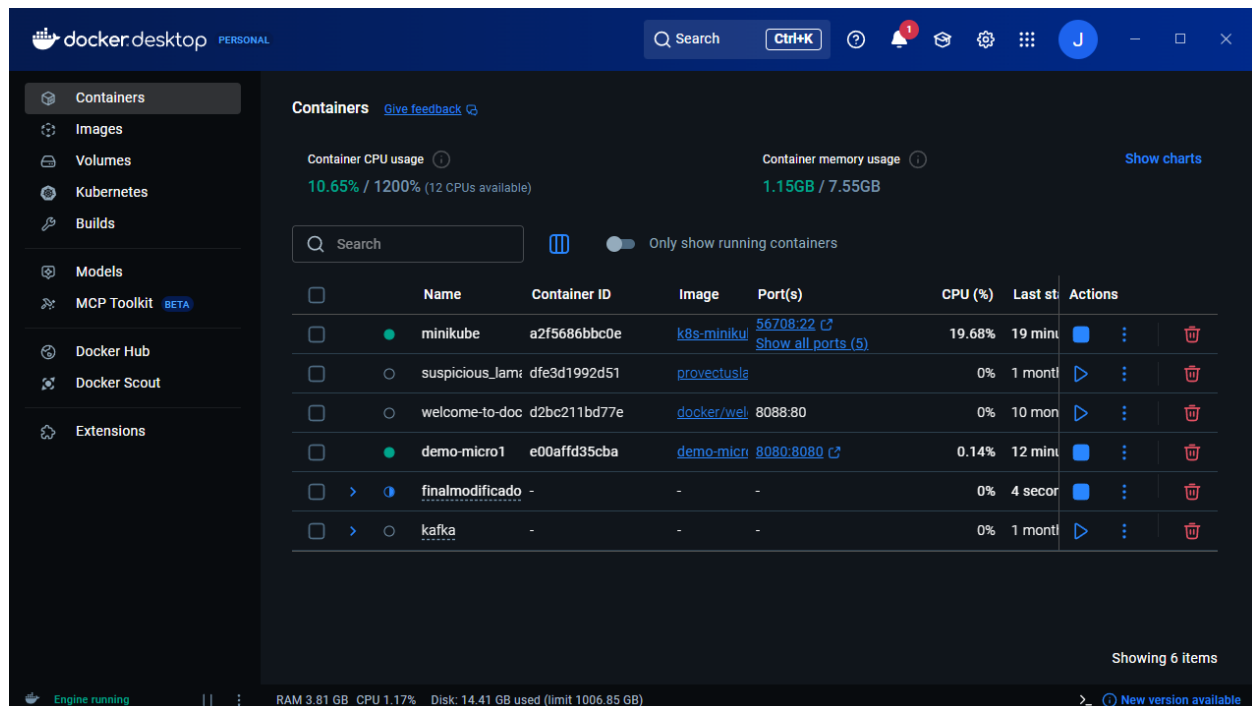
La columna "**Dónde está la evidencia**" indica la sección de este documento y el número de captura de pantalla que respaldan cada entregable.

#	Entregable	Dónde está la evidencia
1	URL del repositorio	Sección 9 · Captura 12 — github.com/Jhont3/micro-calc-k8s
2	Dockerfile funcional	Secciones 3 y 4 · Capturas 3 y 4 (build y ejecución validados)
3	Imagen publicada en Docker Hub	Sección 5 · Captura 5 — jhont3/demo-micro:1
4	Carpeta k8s/ con manifiestos YAML	Sección 6 (namespace, configmap, deployment, service, hpa)
5	Evidencias de ejecución local con Docker	Sección 4 · Captura 4
6	Evidencias de despliegue en Minikube	Secciones 6 y 7 · Capturas 6 a 8
7	Evidencias de prueba del microservicio	Sección 8 · Capturas 9 a 11
8	PDF final con capturas y comandos	Este documento

Sobre el repositorio: GitHub no permite forks privados de repositorios públicos, por lo que se creó un repositorio nuevo (**Jhont3/micro-calc-k8s**, hoy público) a partir del proyecto base **gmacastil/micro-calc**, con atribución en el README. El código se tradujo al inglés (clases, métodos y endpoints: /add, /subtract, /divide) y se agregó Spring Boot Actuator para los probes de Kubernetes. Las decisiones de diseño están documentadas como ADRs en docs/adr/.

1. Verificación de prerequisites

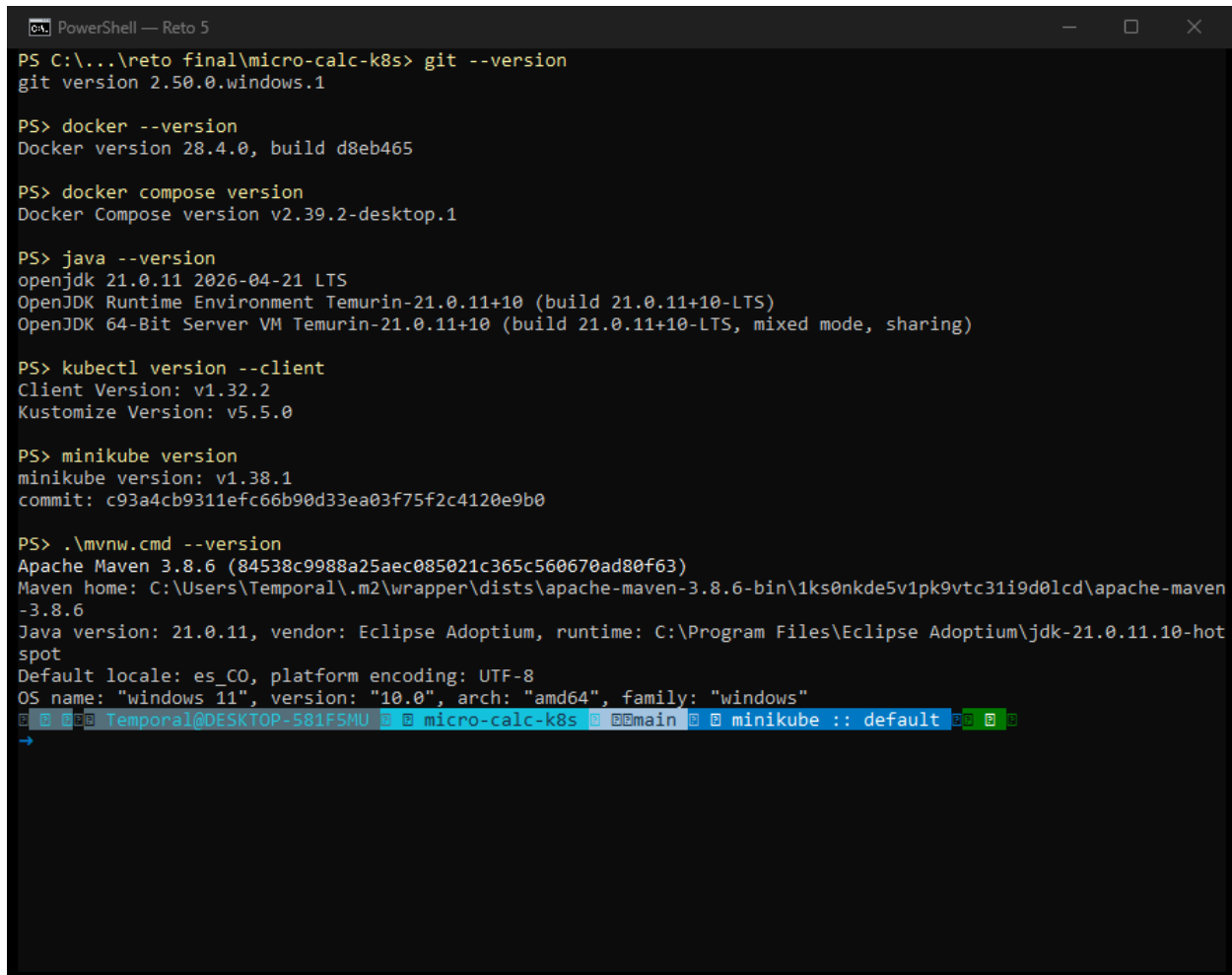
Docker Desktop 4.46 (backend WSL2) en estado **Engine running**; en la lista de contenedores se ven **minikube** (cluster) y **demo-micro1** (contenedor local del microservicio) en ejecución.



Captura 1 — Docker Desktop en estado Running, con los contenedores minikube y demo-micro1.

Verificación sugerida por la guía, ejecutada en PowerShell — Git 2.50, Docker 28.4.0, Docker Compose v2.39.2, JDK 21.0.11 (Temurin), kubectl v1.32.2, Minikube v1.38.1 y Maven 3.8.6 vía wrapper del proyecto (mvnw.cmd):

```
git --version
docker --version
docker compose version
java --version
kubectl version --client
minikube version
.\mvnw.cmd --version
```



```
PowerShell — Reto 5
PS C:\...\reto final\micro-calc-k8s> git --version
git version 2.50.0.windows.1

PS> docker --version
Docker version 28.4.0, build d8eb465

PS> docker compose version
Docker Compose version v2.39.2-desktop.1

PS> java --version
openjdk 21.0.11 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-21.0.11+10 (build 21.0.11+10-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.11+10 (build 21.0.11+10-LTS, mixed mode, sharing)

PS> kubectl version --client
Client Version: v1.32.2
Kustomize Version: v5.5.0

PS> minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0

PS> .\mvnw.cmd --version
Apache Maven 3.8.6 (84538c9988a25aec085021c365c560670ad80f63)
Maven home: C:\Users\Temporal\m2\wrapper\dists\apache-maven-3.8.6-bin\1ks0nkde5v1pk9vtc31i9d0lcd\apache-maven-3.8.6
Java version: 21.0.11, vendor: Eclipse Adoptium, runtime: C:\Program Files\Eclipse Adoptium\jdk-21.0.11-hotspot
Default locale: es_CO, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
Temporal@DESKTOP-581F5MU micro-calc-k8s main minikube :: default
```

Captura 2 — Versiones de todas las herramientas requeridas.

2. Compilación del proyecto

El jar se genera con el Maven Wrapper incluido en el repositorio (la guía permite usar el wrapper en lugar de Maven instalado). La suite de pruebas también fue ejecutada en verde (`.\mvnw.cmd test`).

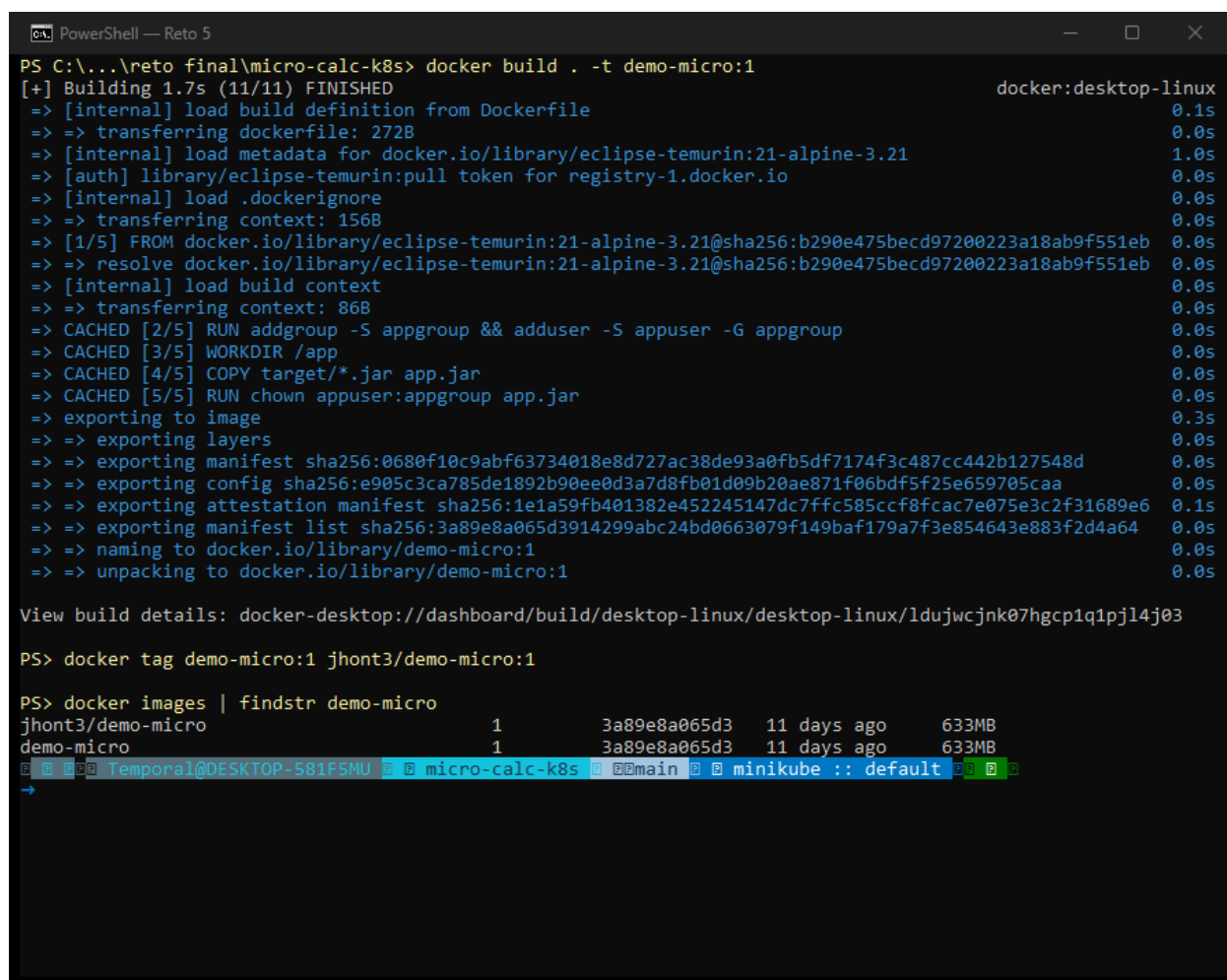
```
.\mvnw.cmd clean package -DskipTests
# genera target\demo-micro-0.0.1-SNAPSHOT.jar
```

3. Dockerfile y construcción de la imagen

Dockerfile single-stage sobre `eclipse-temurin:21-alpine`, con usuario `no-root` (`appuser`) y puerto 8080 expuesto. Se construye la imagen local y se etiqueta para Docker Hub:

```
FROM eclipse-temurin:21-alpine-3.21
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY target/*.jar app.jar
RUN chown appuser:appgroup app.jar
USER appuser
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]

docker build . -t demo-micro:1
docker tag demo-micro:1 jhont3/demo-micro:1
docker images | findstr demo-micro
```



```
PS C:\...\reto final\micro-calc-k8s> docker build . -t demo-micro:1
[+] Building 1.7s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 272B                                0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-alpine-3.21 1.0s
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 156B                                    0.0s
=> [1/5] FROM docker.io/library/eclipse-temurin:21-alpine-3.21@sha256:b290e475becd97200223a18ab9f551eb 0.0s
=> => resolve docker.io/library/eclipse-temurin:21-alpine-3.21@sha256:b290e475becd97200223a18ab9f551eb 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 86B                                     0.0s
=> CACHED [2/5] RUN addgroup -S appgroup && adduser -S appuser -G appgroup 0.0s
=> CACHED [3/5] WORKDIR /app                                     0.0s
=> CACHED [4/5] COPY target/*.jar app.jar                        0.0s
=> CACHED [5/5] RUN chown appuser:appgroup app.jar               0.0s
=> exporting to image                                           0.3s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:0680f10c9abf63734018e8d727ac38de93a0fb5df7174f3c487cc442b127548d 0.0s
=> => exporting config sha256:e905c3ca785de1892b90ee0d3a7d8fb01d09b20ae871f06bdf5f25e659705caa 0.0s
=> => exporting attestation manifest sha256:1e1a59fb401382e452245147dc7ffc585ccf8fcac7e075e3c2f31689e6 0.1s
=> => exporting manifest list sha256:3a89e8a065d3914299abc24bd0663079f149baf179a7f3e854643e883f2d4a64 0.0s
=> => naming to docker.io/library/demo-micro:1                   0.0s
=> => unpacking to docker.io/library/demo-micro:1                 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ldujwcjnk07hgcp1q1pj14j03

PS> docker tag demo-micro:1 jhont3/demo-micro:1

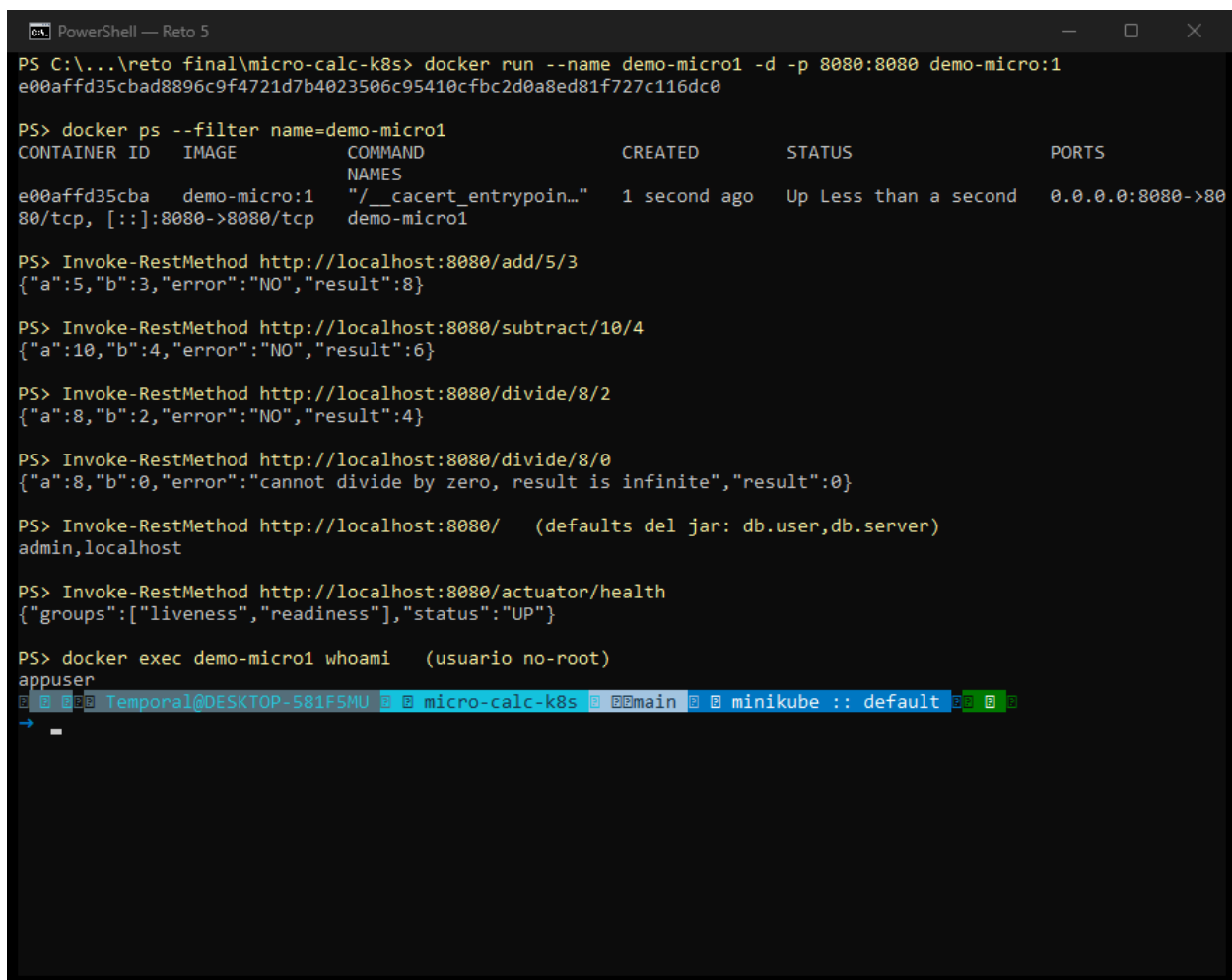
PS> docker images | findstr demo-micro
jhont3/demo-micro          1          3a89e8a065d3      11 days ago      633MB
demo-micro                1          3a89e8a065d3      11 days ago      633MB
Temporal@DESKTOP-581FSMU  micro-calc-k8s  main            minikube :: default
```

Captura 3 — docker build exitoso (11/11 pasos) y la imagen etiquetada como `jhont3/demo-micro:1`.

4. Ejecución local con Docker y pruebas

El contenedor se levanta publicando el puerto 8080 y se prueban todos los endpoints. La división por cero responde con mensaje de error controlado en lugar de fallar; el endpoint raíz muestra los valores por defecto del jar (admin,localhost) — dato clave para el contraste con Kubernetes en la sección 8. docker exec ... whoami confirma que el proceso corre como usuario no-root (appuser).

```
docker run --name demo-micro1 -d -p 8080:8080 demo-micro:1
docker ps --filter name=demo-micro1
Invoke-RestMethod http://localhost:8080/add/5/3
Invoke-RestMethod http://localhost:8080/subtract/10/4
Invoke-RestMethod http://localhost:8080/divide/8/2
Invoke-RestMethod http://localhost:8080/divide/8/0
Invoke-RestMethod http://localhost:8080/
Invoke-RestMethod http://localhost:8080/actuator/health
docker exec demo-micro1 whoami
```



```
PS C:\...\reto final\micro-calc-k8s> docker run --name demo-micro1 -d -p 8080:8080 demo-micro:1
e00affd35cbad8896c9f4721d7b4023506c95410cfbc2d0a8ed81f727c116dc0

PS> docker ps --filter name=demo-micro1
CONTAINER ID   IMAGE          COMMAND
NAME          CREATED        STATUS        PORTS
e00affd35cba   demo-micro:1   "/__cacert_entrypoin..." 1 second ago   Up Less than a second   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
demo-micro1

PS> Invoke-RestMethod http://localhost:8080/add/5/3
{"a":5,"b":3,"error":"NO","result":8}

PS> Invoke-RestMethod http://localhost:8080/subtract/10/4
{"a":10,"b":4,"error":"NO","result":6}

PS> Invoke-RestMethod http://localhost:8080/divide/8/2
{"a":8,"b":2,"error":"NO","result":4}

PS> Invoke-RestMethod http://localhost:8080/divide/8/0
{"a":8,"b":0,"error":"cannot divide by zero, result is infinite","result":0}

PS> Invoke-RestMethod http://localhost:8080/ (defaults del jar: db.user,db.server)
admin,localhost

PS> Invoke-RestMethod http://localhost:8080/actuator/health
{"groups":["liveness","readiness"],"status":"UP"}

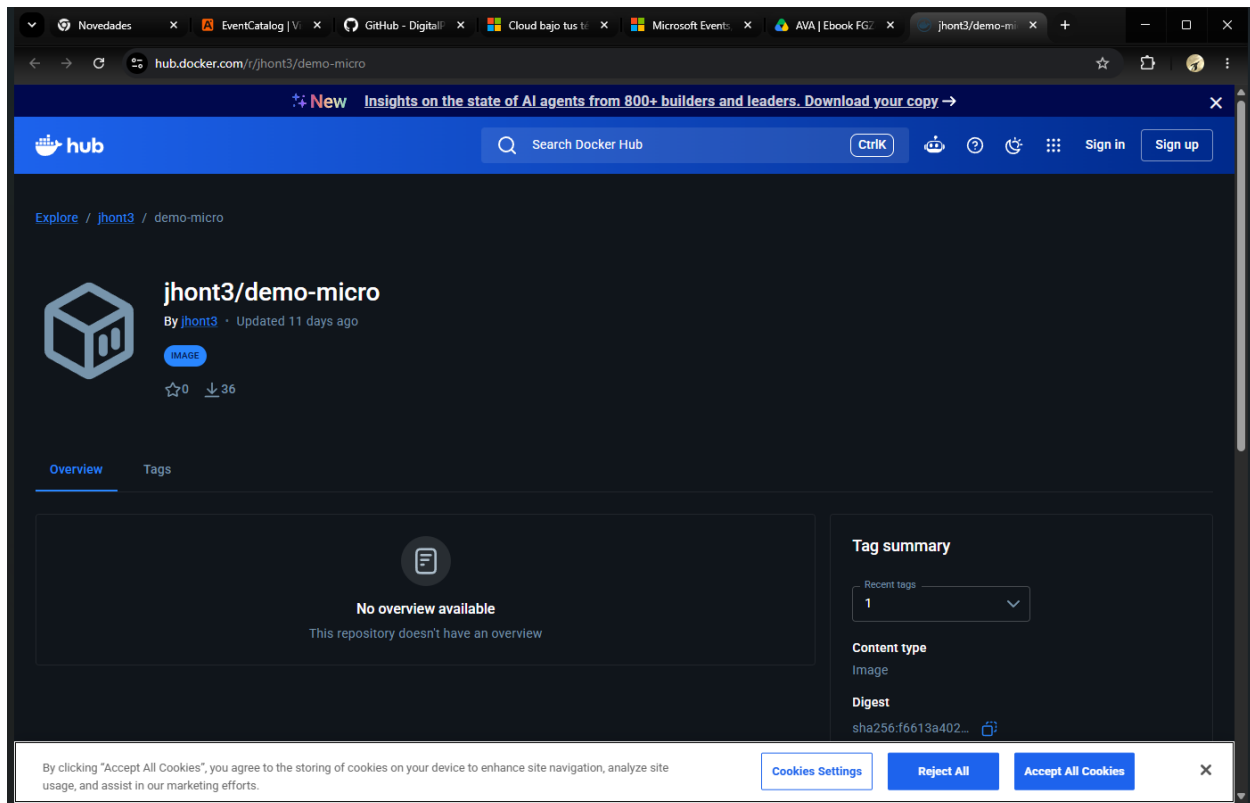
PS> docker exec demo-micro1 whoami (usuario no-root)
appuser
```

Captura 4 — Contenedor corriendo y los 6 endpoints respondiendo correctamente en local.

5. Publicación en Docker Hub

La imagen fue publicada como repositorio público **jhont3/demo-micro** con el tag **1** (digest sha256:f6613a402..., 212 MB). Al ser pública, el cluster puede descargarla sin imagePullSecrets.

```
docker login
docker push jhont3/demo-micro:1
```

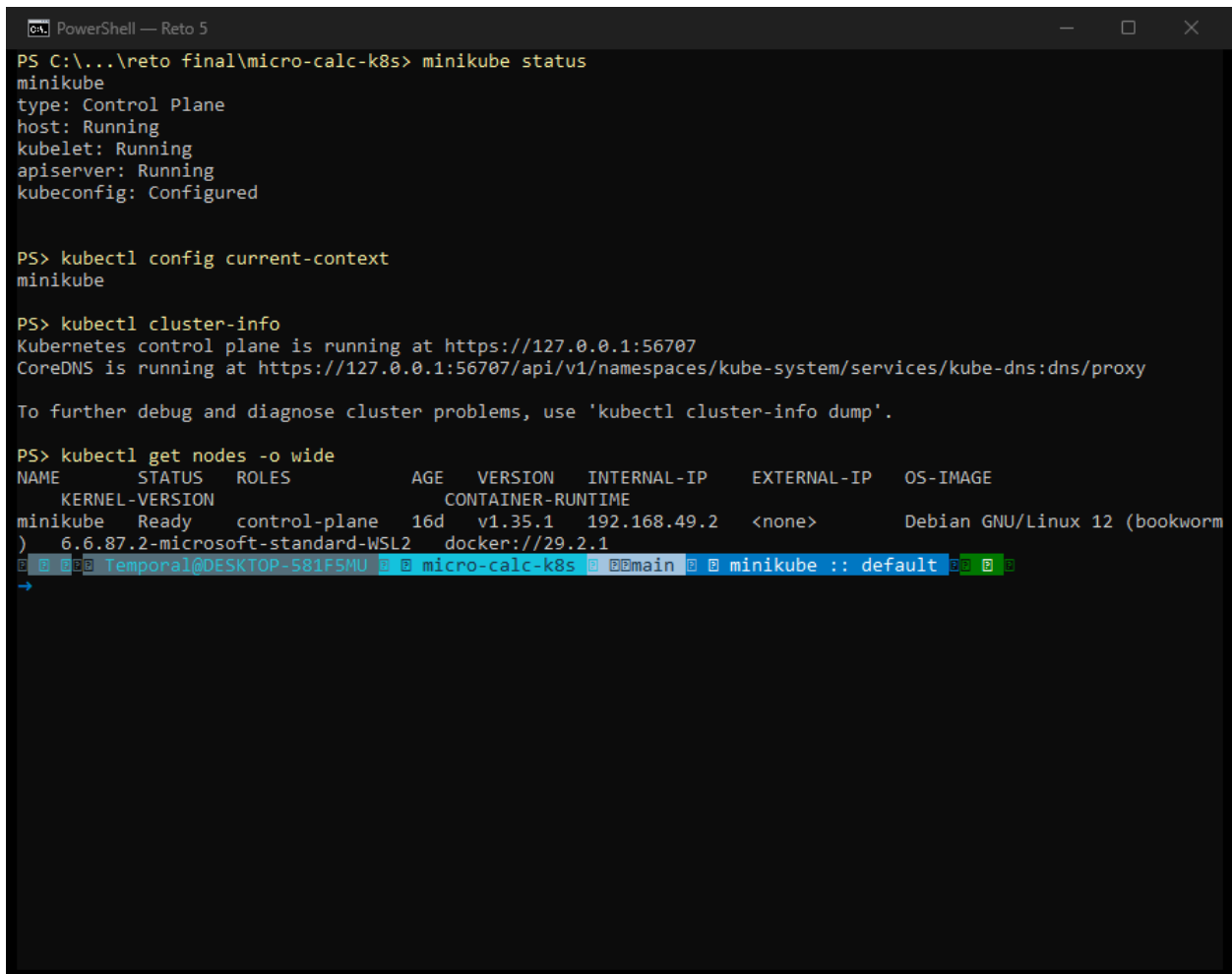


Captura 5 — Página del repositorio en Docker Hub: tag 1, digest sha256:f6613a402..., actualizado hace 11 días.

6. Manifiestos de Kubernetes y arranque de Minikube

La carpeta **k8s/** en la raíz del repo contiene los manifiestos: **00-namespace.yaml** (namespace reto5), **configmap.yaml** (DB_SERVER/DB_USER con valores deliberadamente distintos a los defaults del jar, para demostrar la configuración externalizada), **deployment.yaml** (imagen jhont3/demo-micro:1, probes de liveness/readiness contra /actuador/health, requests/limits de CPU y memoria), **service.yaml** (NodePort 9080→8080; con el driver Docker en Windows, minikube service requiere NodePort) y **hpa.yaml** (1–3 réplicas al 70% de CPU). El cluster se arranca con el driver Docker y el contexto activo de kubectl es **minikube**:

```
minikube start --driver=docker --cpus=2 --memory=4000
minikube status
kubectl config current-context
kubectl cluster-info
kubectl get nodes -o wide
```



```
PS C:\...\reto final\micro-calc-k8s> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

PS> kubectl config current-context
minikube

PS> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:56707
CoreDNS is running at https://127.0.0.1:56707/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.

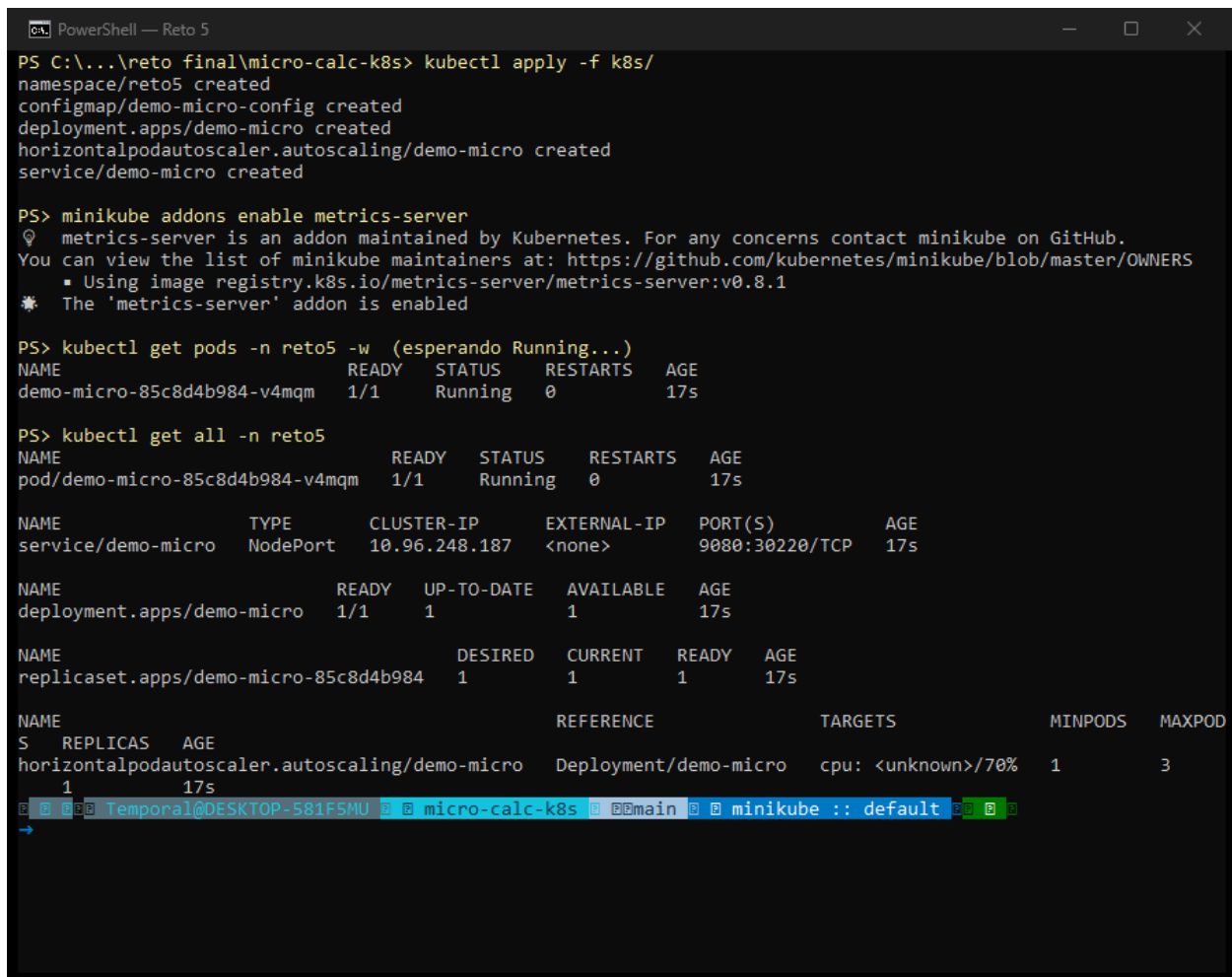
PS> kubectl get nodes -o wide
NAME              STATUS    ROLES          AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION    CONTAINER-RUNTIME
minikube          Ready     control-plane  16d   v1.35.1   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2  docker://29.2.1
```

Captura 6 — Minikube corriendo (host/kubelet/apiserver Running), contexto minikube, nodo Ready v1.35.1.

7. Despliegue en Minikube

Un solo `kubectl apply -f k8s/` crea los 5 recursos (la carpeta es autocontenida gracias a `00-namespace.yaml`). Se habilita `metrics-server` para el HPA. El pod queda 1/1 Running en ~17 segundos, descargando la imagen publicada desde Docker Hub — prueba de que la imagen publicada funciona por sí sola:

```
kubectl apply -f k8s/
minikube addons enable metrics-server
kubectl get pods -n reto5
kubectl get all -n reto5
```



```
PS C:\...\reto final\micro-calc-k8s> kubectl apply -f k8s/
namespace/reto5 created
configmap/demo-micro-config created
deployment.apps/demo-micro created
horizontalpodautoscaler.autoscaling/demo-micro created
service/demo-micro created

PS> minikube addons enable metrics-server
ⓘ metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  ■ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
⚡ The 'metrics-server' addon is enabled

PS> kubectl get pods -n reto5 -w (esperando Running...)
NAME                                READY   STATUS    RESTARTS   AGE
demo-micro-85c8d4b984-v4mqm         1/1     Running   0          17s

PS> kubectl get all -n reto5
NAME                                READY   STATUS    RESTARTS   AGE
pod/demo-micro-85c8d4b984-v4mqm     1/1     Running   0          17s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
service/demo-micro                  NodePort      10.96.248.187 <none>        9080:30220/TCP   17s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/demo-micro           1/1     1            1          17s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/demo-micro-85c8d4b984 1         1         1       17s

NAME                                REFERENCE          TARGETS          MINPODS   MAXPOD
S   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/demo-micro  Deployment/demo-micro  cpu: <unknown>/70%  1         3
1       17s
```

Captura 7 — Recursos creados y pod demo-micro 1/1 Running; Service NodePort 9080:30220; HPA activo.

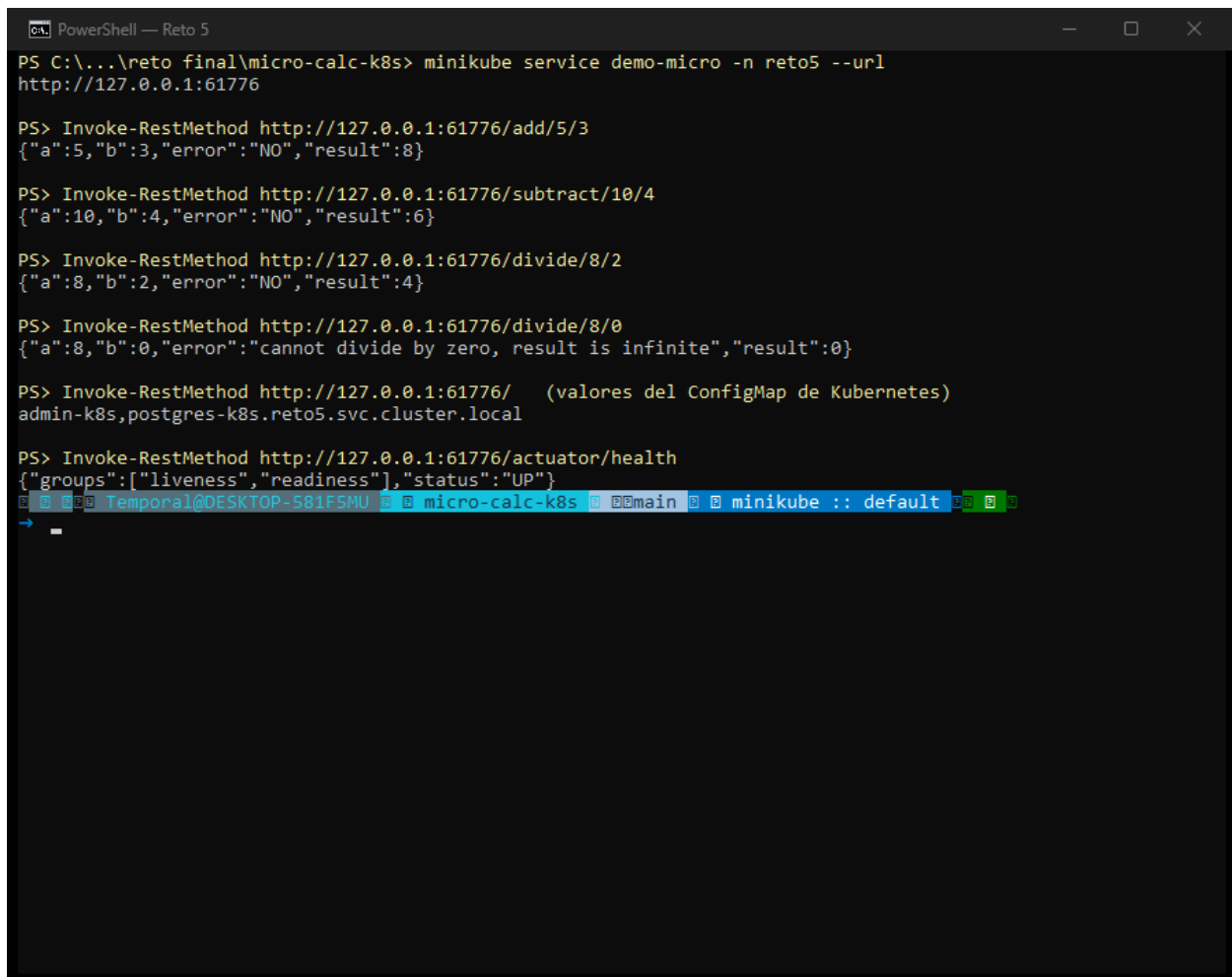
El HPA reporta consumo real (cpu: 4%/70%) una vez que `metrics-server` está arriba, y los logs del deployment muestran Spring Boot 4.1.0 arrancando en el puerto 8080 con PID 1 ejecutado por `appuser`:

```
kubectl get hpa -n reto5
kubectl logs -n reto5 deployment/demo-micro --tail=25
```


8. Pruebas del microservicio en Minikube

minikube service genera un túnel local hacia el NodePort. Se prueban los mismos 6 endpoints contra el cluster; todos responden igual que en local, excepto el endpoint raíz, que ahora devuelve **admin-k8s,postgres-k8s.reto5.svc.cluster.local** — los valores del ConfigMap, no los del jar. Misma imagen inmutable, comportamiento distinto según el entorno: configuración externalizada de extremo a extremo.

```
minikube service demo-micro -n reto5 --url
# -> http://127.0.0.1:61776 (puerto aleatorio del túnel)
Invoke-RestMethod http://127.0.0.1:61776/add/5/3
Invoke-RestMethod http://127.0.0.1:61776/divide/8/0
Invoke-RestMethod http://127.0.0.1:61776/
Invoke-RestMethod http://127.0.0.1:61776/actuator/health
```



```
PS C:\...\reto final\micro-calc-k8s> minikube service demo-micro -n reto5 --url
http://127.0.0.1:61776

PS> Invoke-RestMethod http://127.0.0.1:61776/add/5/3
{"a":5,"b":3,"error":"NO","result":8}

PS> Invoke-RestMethod http://127.0.0.1:61776/subtract/10/4
{"a":10,"b":4,"error":"NO","result":6}

PS> Invoke-RestMethod http://127.0.0.1:61776/divide/8/2
{"a":8,"b":2,"error":"NO","result":4}

PS> Invoke-RestMethod http://127.0.0.1:61776/divide/8/0
{"a":8,"b":0,"error":"cannot divide by zero, result is infinite","result":0}

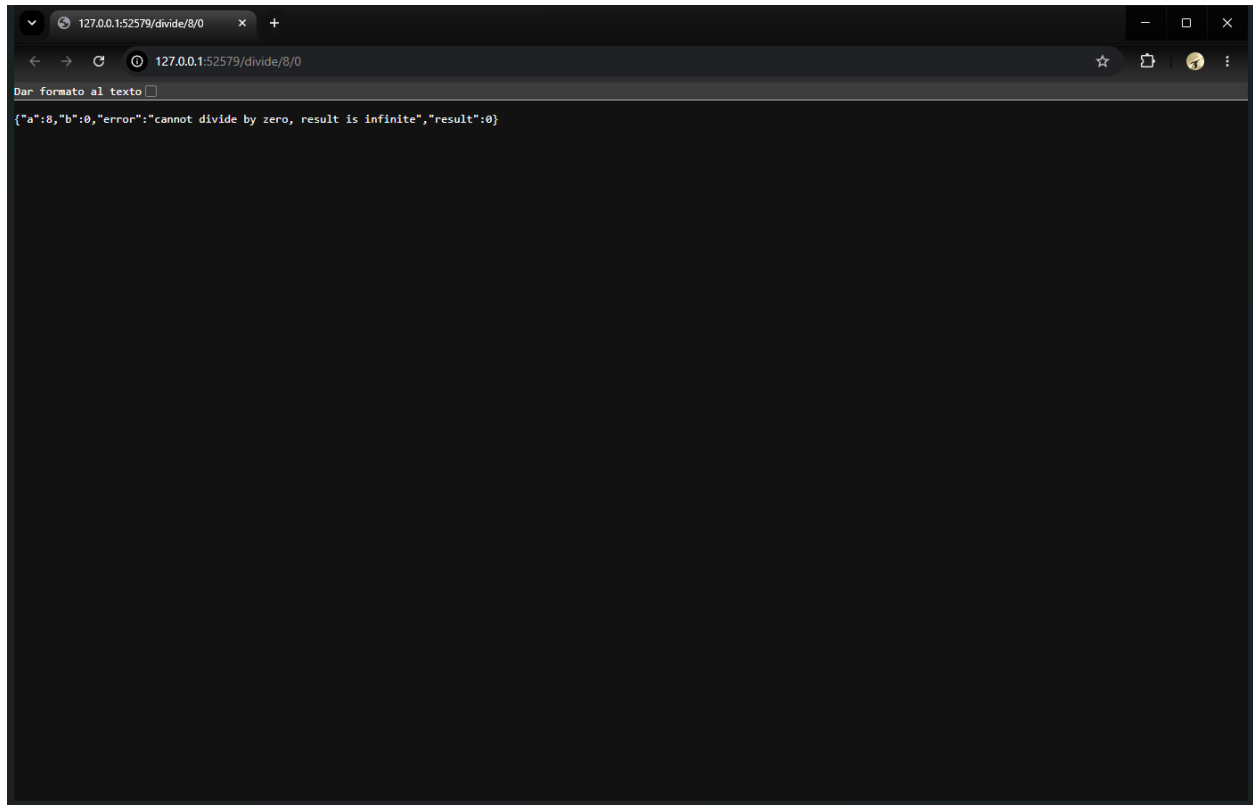
PS> Invoke-RestMethod http://127.0.0.1:61776/ (valores del ConfigMap de Kubernetes)
admin-k8s,postgres-k8s.reto5.svc.cluster.local

PS> Invoke-RestMethod http://127.0.0.1:61776/actuator/health
{"groups":["liveness","readiness"],"status":"UP"}

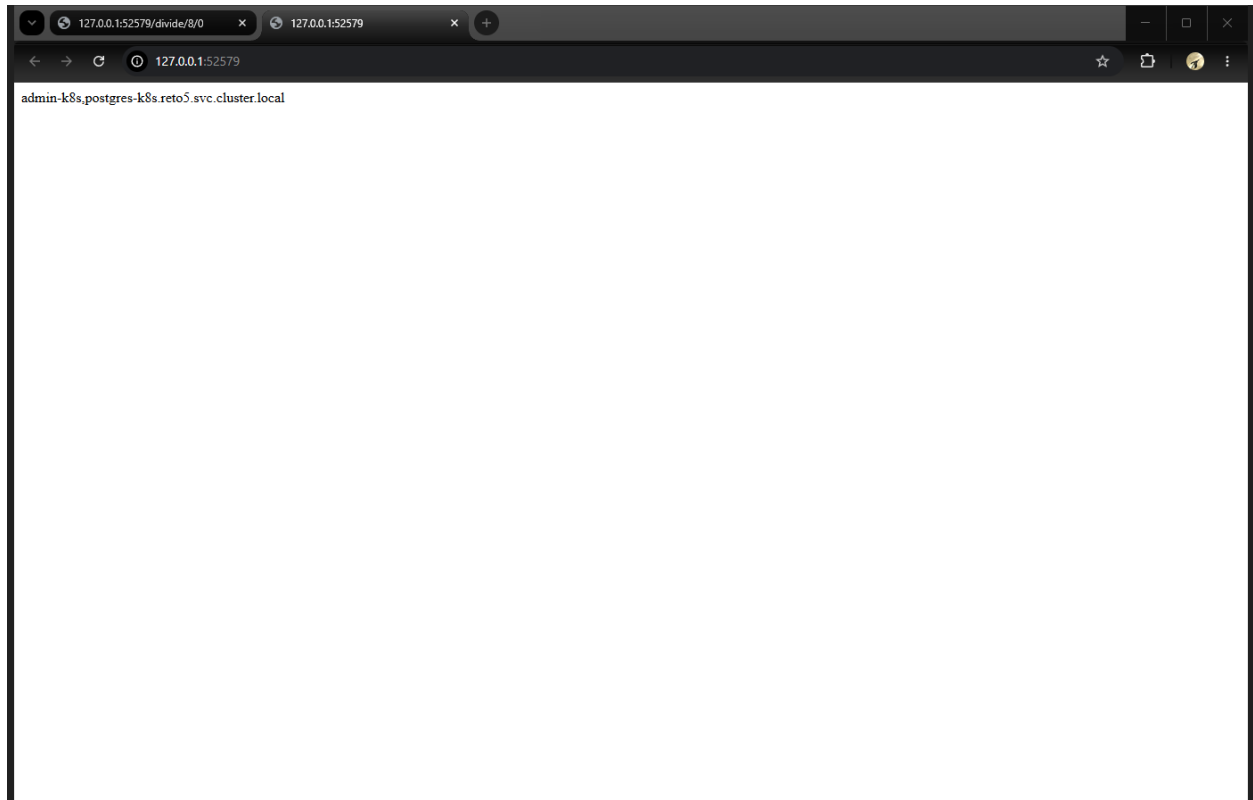
[1] [2] [3] Temporal@DESKTOP-581F5MU [4] micro-calc-k8s [5] main [6] minikube :: default [7] [8] [9]
```

Captura 9 — Todos los endpoints probados desde PowerShell contra el servicio en Minikube.

Las mismas pruebas desde el navegador (Chrome) contra la URL del túnel de Minikube:



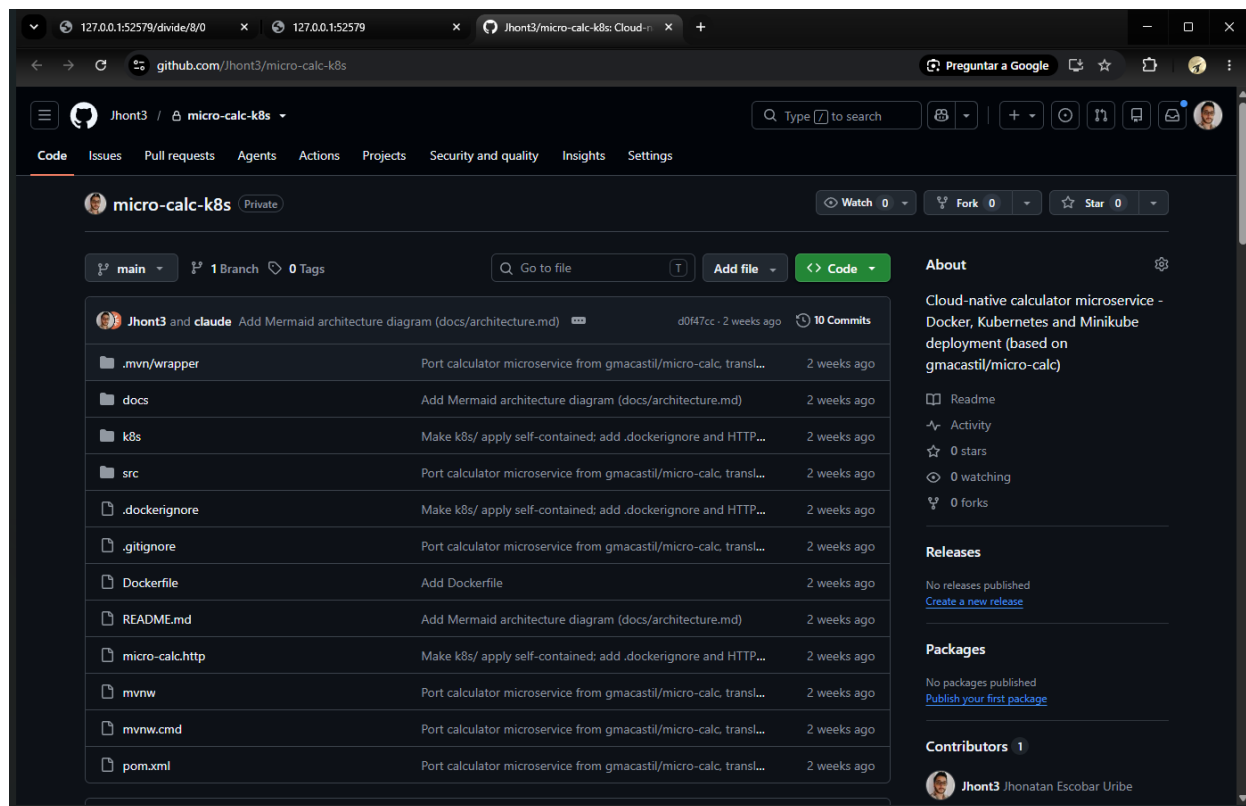
Captura 10 — /divide/8/0 desde el navegador: manejo controlado de la división por cero.



Captura 11 — Endpoint raíz desde el navegador mostrando los valores inyectados por el ConfigMap.

9. Repositorio en GitHub

Código, Dockerfile, manifiestos k8s/ y documentación (spec, plan, ADRs, reporte y diagrama de arquitectura Mermaid) publicados en github.com/Jhont3/micro-calc-k8s (rama main). El repositorio es **público**, por lo que la URL basta para revisarlo; el README incluye el diagrama de arquitectura del sistema.



Captura 12 — Repositorio Jhont3/micro-calc-k8s en GitHub con todo el contenido en main.

10. Conclusiones

- Se completó el ciclo Cloud Native de extremo a extremo: código → imagen Docker → registro público (Docker Hub) → despliegue declarativo en Kubernetes (Minikube) → validación por HTTP.
- La imagen que corre en el cluster es exactamente la publicada en Docker Hub (el pod la descargó del registro, no del build local), lo que valida el flujo de publicación.
- El contraste del endpoint raíz entre Docker local (admin,localhost) y Kubernetes (admin-k8s,postgres-k8s.reto5.svc.cluster.local) demuestra la configuración externalizada vía ConfigMap sin reconstruir la imagen.
- Los probes de liveness/readiness contra /actuador/health, los límites de recursos y el HPA (1–3 réplicas, 70% CPU) dejan el despliegue en una configuración operable, no solo demostrativa.
- El contenedor corre como usuario no-root (appuser), verificado con docker exec whoami.